Employee Attrition Classification

Fredy Paz
University Boulder Colorado
Boulder, CO
fredy.paz@colorado.edu

Abstract

Employee attrition presents significant operational and financial challenges for organizations. This study applies data mining and machine learning techniques to predict employee attrition using a Human Resources dataset containing 1,470 records and 35 features. The workflow includes data preprocessing, feature engineering, class imbalance mitigation, and model evaluation. Both unsupervised methods (Apriori and association rule mining) and supervised classification models including Logistic Regression, Decision Trees, Random Forest, XGBoost, and AdaBoost were implemented. Special attention was given to handling class imbalance through threshold optimization, class weighting, and SMOTE. Model performance was evaluated using recall, PR-AUC, ROC-AUC, and confusion matrices, with emphasis on maximizing recall to reduce missed attrition cases. Ensemble methods, particularly AdaBoost and Random Forest, achieved the strongest balance between recall and overall accuracy. Key predictive features consistently included overtime, total working years, job level, and promotion history. The results demonstrate that ensemble learning combined with threshold optimization provides an effective approach for identifying employees at risk of attrition.

1. Introduction

This study applies data mining and machine learning techniques to classify and predict employee attrition. The dataset used contains HR employee data within a business organization. The dataset was retrieved from Kaggle [1] . It is made up of 35 columns (features) with 1470 rows (employees/data entries). The analysis includes data preprocessing, feature engineering, class imbalance handling, and implementation of both unsupervised and supervised learning methods. Models evaluated include Logistic Regression, Decision Trees, Random Forest, XGBoost, and AdaBoost, alongside association rule mining for behavioral pattern discovery.

The main contributions of this work include a comprehensive comparison of class imbalance handling techniques, evaluation of multiple machine learning models using recall-focused metrics, and identification of key factors consistently associated with attrition risk. The findings provide both predictive insights and practical implications for workforce retention strategies.

2. Related Work

There has been a significant amount of machine learning and data mining techniques applied to employee attrition prediction recently. In a

recent research paper published in February 2026 “Integrating machine learning and explainable AI for employee attrition prediction in HR analytics” [2] Al-Ali et al. proposed an end-to-end framework similar to what we aim to do in this report. The paper integrates preprocessing, feature selection, hyperparameter optimization, and explainability tools such as SHAP to improve both predictive performance and interpretability. Their approach evaluates multiple machine learning models on several HR datasets and demonstrates strong performance, particularly using ensemble methods such as Adaptive Boosting and Histogram Gradient Boosting. Additionally, explainability analysis identified key factors influencing attrition, including overtime, job satisfaction, and job level, enabling more actionable organizational insights. The paper also highlights several related studies using employee attrition data. Predominantly, the works related use IBM HR Data. The methods used vary per study but several of them applied methods and models which we will see in this report. Some of the most popular are Logistic Regression, Random Forest, and Decision Tree. Accuracy ranges from 75% up to 99%. [2] We intend to reference these related works to compare the performance of our methods and results. Unlike prior studies that primarily focus on accuracy, this study emphasizes recall optimization and threshold tuning to better identify employees at risk.

3. Proposed Work

This study applies data mining and machine learning techniques to predict employee attrition using methods introduced in prior DTSA coursework. Python was used as the primary programming language, along with commonly used libraries for data manipulation, visualization, and modeling, including Pandas, NumPy, Scikit-learn, and Matplotlib.

The analysis began with data understanding and exploration to ensure familiarity with the dataset’s structure, feature types, and potential data quality issues. Particular attention was given to identifying whether features were categorical, binary, continuous, or discrete, as this informed appropriate preprocessing techniques. Data preparation steps included cleaning inconsistent entries, encoding categorical variables, and addressing multicollinearity.

Following preprocessing, both unsupervised and supervised data mining methods were implemented. Unsupervised techniques included correlation analysis, frequent pattern mining, and association rule discovery to identify relationships among employee characteristics. Supervised learning models included Logistic Regression, Decision Trees, and ensemble methods such as Random Forest and boosting algorithms.

Each model was evaluated using multiple performance metrics to determine its effectiveness in predicting employee attrition. Special consideration was given to class imbalance and its impact on model performance. The results were then compared to identify the most appropriate predictive approach for this dataset.

3.1. Data Understanding and Pre-Processing

The work was started by understanding the dataset. As shown in Figure 1, the dataset is composed of 34 features of different data types. An initial finding is the higher count of values for Age and Attrition. After exploring those specific items, I found there were footer notes appearing in those 2 columns. See example figure 2 for Attrition. Those rows were removed

and we continue reviewing the rest of the data set to ensure consistency and completeness.

```
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1476 non-null   object
 1   Attrition                 1476 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   float64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   float64
 6   Education                 1470 non-null   float64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   float64
 9   EmployeeNumber            1470 non-null   float64
 10  EnvironmentSatisfaction   1470 non-null   float64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   float64
 13  JobInvolvement            1470 non-null   float64
 14  JobLevel                  1470 non-null   float64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   float64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   float64
 19  MonthlyRate               1470 non-null   float64
 20  NumCompaniesWorked        1470 non-null   float64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   float64
 24  PerformanceRating         1470 non-null   float64
 25  RelationshipSatisfaction  1470 non-null   float64
 26  StandardHours             1470 non-null   float64
 27  StockOptionLevel          1470 non-null   float64
 28  TotalWorkingYears         1470 non-null   float64
 29  TrainingTimesLastYear     1470 non-null   float64
 30  WorkLifeBalance           1470 non-null   float64
 31  YearsAtCompany            1470 non-null   float64
 32  YearsInCurrentRole        1470 non-null   float64
 33  YearsSinceLastPromotion   1470 non-null   float64
 34  YearsWithCurrManager      1470 non-null   float64
```

Figure 1

```
Attrition
No        1233
Yes        237
 10:07       1
 15:16       1
 12:15       1
 14:15       1
 10:09       1
 16:24       1
Name: count, dtype: int64
```

Figure 2

Next, we review our target feature, Attrition, for imbalance. This type of data tends to be organically imbalanced as a healthy business should have a low % of attrition. According to HR publication SHRM, the ideal rate should be below 10% [4]. In Figure 3, we can see the ratio for this target feature is 83.9% No Attrition and 16.1% for Attrition. This is a class imbalance that must be factored in our analysis.

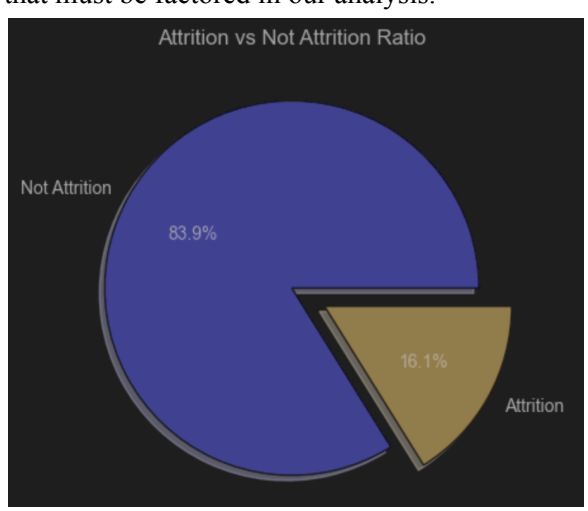

Figure 3

Reviewing for irrelevant features is also important. For example, Employee Count,

Standard Hours, and Over 18 contained the same values across all rows. Employee Number acts as ID number which is also irrelevant and introduces noise during training stages.

I also found that the dataset contained non-numeric features which can be changed to binary. The features changed to binary are Attrition and Overtime. They were changed from No/Yes to 0/1.

Additionally, there are several categorical columns that needed to be one hot encoded. Using the library One Hot Encoder from SciKit Learn, we are able to change categorical features to binary. For example, Marital Status with values Divorced, Single, Married changes to 3 separate columns MaritalStatus_Divorced, MaritalStatus_Single, MaritalStatus_Married with values 0 or 1. Encoding this way allows better compatibility with machine learning models while allowing non numeric data to be ingested. [5]

Lastly, a correlation heatmap was created to identify features with high multicollinearity, Figure 4. This review revealed that Gender_Male and Gender_Female were 100% correlated. This is due to the columns being mirrors of each other after one hot coding. One of the columns can be removed as one alone can serve as a binary feature for both.

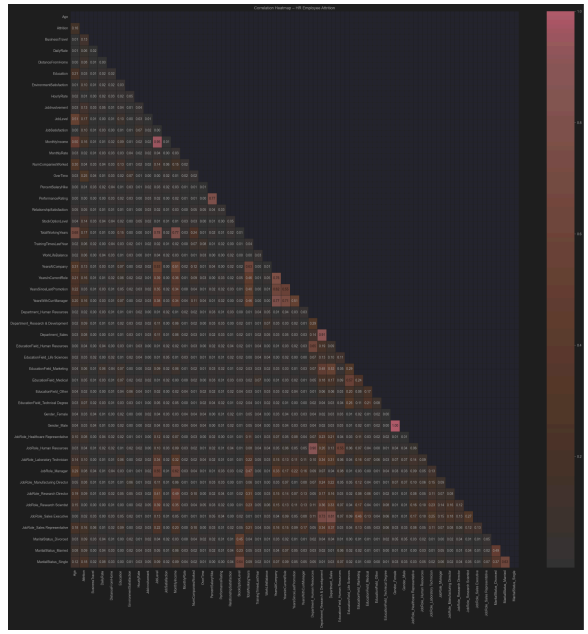

Figure 4

3.2. Unsupervised Learning

3.2.1 Frequent Patter Analysis

Once we have sufficient data understanding and pre-processing is complete, we can move to training. The first method used is Frequent Pattern Analysis, Apriori algorithm. This algorithm "...has been designed to operate on databases containing transactions, such as purchases by customers of a store. An itemset is considered as "frequent" if it meets a user-specified support threshold. For instance, if the support threshold is set to 0.5 (50%), a frequent itemset is defined as a set of items that occur together in at least 50% of all transactions in the database." Since this algorithm is checking for item occurrences, the features must be binary, they either occur or not [6]. Therefore, only features that are binary or have been one hot coded are used. See figure 5 for list of

features.

```
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   Attrition                           1470 non-null   int64
 1   OverTime                            1470 non-null   int64
 2   Department_Human Resources          1470 non-null   float64
 3   Department_Research & Development   1470 non-null   float64
 4   Department_Sales                    1470 non-null   float64
 5   EducationField_Human Resources      1470 non-null   float64
 6   EducationField_Life Sciences        1470 non-null   float64
 7   EducationField_Marketing            1470 non-null   float64
 8   EducationField_Medical              1470 non-null   float64
 9   EducationField_Other                1470 non-null   float64
 10  EducationField_Technical Degree     1470 non-null   float64
 11  Gender_Female                       1470 non-null   float64
 12  JobRole_Healthcare Representative   1470 non-null   float64
 13  JobRole_Human Resources             1470 non-null   float64
 14  JobRole_Laboratory Technician       1470 non-null   float64
 15  JobRole_Manager                     1470 non-null   float64
 16  JobRole_Manufacturing Director      1470 non-null   float64
 17  JobRole_Research Director           1470 non-null   float64
 18  JobRole_Research Scientist          1470 non-null   float64
 19  JobRole_Sales Executive             1470 non-null   float64
 20  JobRole_Sales Representative        1470 non-null   float64
 21  MaritalStatus_Divorced              1470 non-null   float64
 22  MaritalStatus_Married               1470 non-null   float64
 23  MaritalStatus_Single                1470 non-null   float64
```

Figure 5

The default support for apriori is .5, however with this type of dataset, using a lower support will provide more results. For example, with support of 10%, we only get 1 itemset. Figure 6

```
#Code for Apriori
X = data_apriori.astype(bool)
freq_itemsets = apriori(X, min_support=0.5,use_colnames=True)
freq_itemsets = freq_itemsets.sort_values(by='support', ascending=False,ignore_index=True)
print(freq_itemsets)

✓ [270] 13ms

     support                          itemsets
0  0.653741  (Department_Research & Development)
```

Figure 6

With a minimum support of .1, we get more itemsets. See figure 7 for top 10 itemsets. Higher support only shows single items but starts getting more itemsets at 3% or lower.

```
      support                                          itemsets
0    0.653741                 (Department_Research & Development)
1    0.457823                            (MaritalStatus_Married)
2    0.412245                      (EducationField_Life Sciences)
3    0.400000                                     (Gender_Female)
4    0.319728                              (MaritalStatus_Single)
5    0.315646                            (EducationField_Medical)
6    0.303401                                  (Department_Sales)
7    0.299320  (Department_Research & Development, EducationF...
8    0.294558  (Department_Research & Development, MaritalSta...
9    0.282993                                          (OverTime)
10   0.257823  (Department_Research & Development, Gender_Fem...
```

Figure 7

3.2.2 Association Rules

Next, we can implement Association Rules. “An association rule is an implication expression of the form X→Y, where X and Y are disjoint itemsets. A more concrete example based on consumer behaviour would be {Diapers}→{Beer} suggesting that people who buy diapers are also likely to buy beer. To evaluate the "interest" of such an association rule, different metrics have been developed. The current implementation makes use of the confidence and lift metrics.” [7]. For this dataset, we are looking to answer which behavior is closely tied to attrition. We will be looking for lift, representing how more likely they are to leave compared to the average employee, sorted by confidence.

Based on the results from Apriori, we will rerun with a min support of 3% and set a minimum threshold for confidence of 2%.

The results show that the number 1 itemset for employees who leave is Marital Status Single and Overtime. This demographic has an Antecedent Support of 8.9%, which represents the percent of total employees meeting that criteria. The confidence for this itemset is .49 which indicates 49% of these employees leave the company. Finally a lift score of 3.07 tells us that these employees are 3 times more likely to leave than the average employee. Figure 8.

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift
0	(MaritalStatus_Single, OverTime)	(Attrition)	0.089116	0.161224	0.044218	0.496183	3.077592
1	(JobRole_Laboratory Technician)	(Attrition, Department_Research & Development)	0.176190	0.090476	0.042177	0.239382	2.645804
2	(Department_Sales, OverTime)	(Attrition)	0.087075	0.161224	0.032653	0.375000	2.325949
3	(Department_Sales, MaritalStatus_Single)	(Attrition)	0.104082	0.161224	0.036054	0.346405	2.148589
4	(OverTime, EducationField_Life Sciences)	(Attrition)	0.113605	0.161224	0.034694	0.305389	1.894186
5	(OverTime)	(Attrition)	0.282993	0.161224	0.086395	0.305288	1.893561
6	(Department_Research & Development, OverTime)	(Attrition)	0.184354	0.161224	0.050340	0.273063	1.693680
7	(Gender_Female, OverTime)	(Attrition)	0.122449	0.161224	0.031973	0.261111	1.619550
8	(MaritalStatus_Single)	(Attrition)	0.319728	0.161224	0.081633	0.255319	1.583625
9	(MaritalStatus_Single, EducationField_Life Sci...	(Attrition)	0.136735	0.161224	0.033333	0.243781	1.512060
10	(JobRole_Laboratory Technician)	(Attrition)	0.176190	0.161224	0.042177	0.239382	1.484776

Figure 8

3.3 Class Imbalance

Before performing supervised learning, we must address class imbalance. As shown previously, Attrition has a ratio of only 16%. Most models do not perform as well with imbalanced datasets. In order to resolve this, there are several methods available to use, like upsampling (SMOTE), downsampling, or adjusting thresholds. The downside of upsample or downsampling is the introduction of synthetic data. For example, the SMOTE library in python [9] works by increasing the population of the minority class (Attrition in this case) synthetically. Additionally, from research papers reviewed [8], similar results can be achieved by using threshold instead of introducing synthetic data. In order to decide which solution to use for the class imbalance in this dataset, I ran my own testing using Logistic Regression model paired with SMOTE, thresholds, and built in features like class_weight in the Logistic Regression (LR) model in the python library [10]. To determine the best threshold, we will use a Recall target of at least 70% using the library from SciKit Learn, Precision Recall Curve [11]. That is, we will focus on maximizing the amount of True Positives identified by the model. Recall is important to use when missing a positive case (Attrition), is more costly than having a False Positive.

From the testing results shown below, the best option to resolve imbalance and maximize recall, is using LR with an adjusted threshold of .115, see Figure 9. By adjusting the threshold to .115, we can reach the Recall target of at least 70% while minimizing impact on other metrics. Accuracy remains relatively high with 73% and the recall for Class 0 (No Attrition) at 73%. Going forward threshold will be used to resolve class imbalance and maximize identification of True Positives.

LR - standard
ROC-AUC: .808
PR-AUC: .569
Recall: .362
Accuracy: .861

LR - class_weight: Balanced
ROC-AUC: .799
PR- AUC: .541
Recall: .638
Accuracy: .759

LR - SMOTE
ROC-AUC: .733
PR- AUC: .414
Recall: .553
Accuracy: .718

LR - Threshold with 70% recall target
ROC-AUC: .808
PR- AUC: .569
Recall: .72
Accuracy: .73

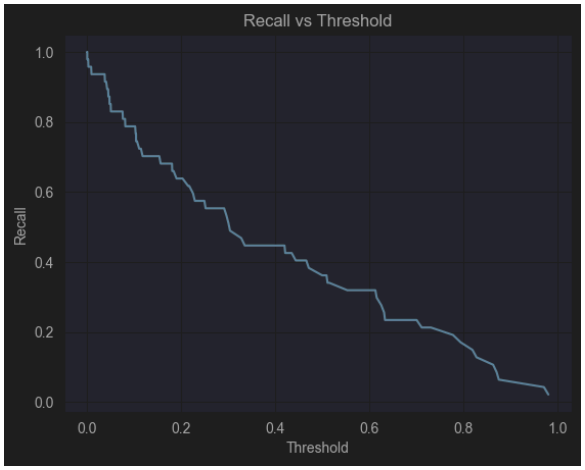

Figure 9

3.4. Supervised Learning

3.4.1 Logistic Regression

Moving on to Supervised Learning, we will review **Logistic Regression**. This model is used to calculate the probability of an input belonging to a specific class. For example, calculating The results for this method can be seen in Figure 10. While Accuracy is one of the fundamental metrics to evaluate results for Machine Learning, the most relevant metric will depend on the goal of the analysis. Further explanations for each metric can be found in the Evaluation section of this paper. For this application, I mostly focused on PR-AUC, Recall, Confusion Matrix, and Accuracy.

With an adjusted threshold of .115, we can get the results shown previously and in figure 10.

```
====================================
Chosen Threshold: 0.115
Target Recall: 0.7
====================================

Confusion Matrix (thresholded):
                   Pred: No Attr  Pred: Attr
Actual: No Attr            181          66
Actual: Attr                13          34

ROC-AUC: 0.8084244982341288
PR-AUC: 0.569651514586499

Classification Report:

              precision    recall  f1-score   support

           0       0.93      0.73      0.82       247
           1       0.34      0.72      0.46        47

    accuracy                           0.73       294
   macro avg       0.64      0.73      0.64       294
weighted avg       0.84      0.73      0.76       294
```

Figure 10

For LR Feature Importance, we see that the top 3 features are Overtime, JobLevel, and YearsSinceLastPromotion. Overtime is especially impactful with a coefficient of .771, meaning that employees who work over time are 2.16 times more likely to leave. We can calculate the odds from the coefficient with the equation below. All features shown in Figure 11.

Odds Ratio=e^0.771763=2.1636

	coefficient	odds_ratio
OverTime	0.771763	2.163577
JobLevel	0.654995	1.925133
YearsSinceLastPromotion	0.490378	1.632934
JobRole_Human Resources	0.472238	1.603580
NumCompaniesWorked	0.460842	1.585408
JobRole_Laboratory Technician	0.455883	1.577565
BusinessTravel	0.455803	1.577440
JobRole_Sales Representative	0.341042	1.406413
DistanceFromHome	0.318005	1.374383
YearsAtCompany	0.257415	1.293581

Figure 11

3.4.2 Random Forest

The next model is **Random Forest.** This algorithm "..uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it as ensemble learning technique. This helps in improving accuracy and reducing errors." [12] This algorithm was implemented using the RandomForestClassifier from SciKit Learn. With a threshold of .188 we can get a Recall for class 1 of 72%, Accuracy of 78%, and a Recall for class 0 of 79%. See full results in Figure 12

```
====================================
Chosen Threshold: 0.188
Target Recall: 0.7
====================================

Confusion Matrix (thresholded):
                   Pred: No Attr  Pred: Attr
Actual: No Attr            195          52
Actual: Attr                13          34

ROC-AUC: 0.791024205357912
PR-AUC: 0.4247317796968544

Classification Report:

              precision    recall  f1-score   support

           0       0.94      0.79      0.86       247
           1       0.40      0.72      0.51        47

    accuracy                           0.78       294
   macro avg       0.67      0.76      0.68       294
weighted avg       0.85      0.78      0.80       294
```

Figure 12

Feature importance for Random Forest yields Monthly Income, Age, and Total Working Years as the top 3 features. Top 10 results in Figure 13.

```
MonthlyIncome          0.082119
Age                    0.075349
TotalWorkingYears      0.070911
OverTime               0.058298
YearsAtCompany         0.050126
DailyRate              0.047028
YearsWithCurrManager   0.042907
DistanceFromHome       0.042771
HourlyRate             0.039669
MonthlyRate            0.037597
```

Figure 13

3.4.3 Decision Trees

Decision Trees are a "non-parametric supervised learning method used for classification and regression. The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. A tree can be seen as a piecewise constant approximation." [13]. Decision Trees are impacted quite differently with imbalance and thresholds. In order to choose the best imbalance solution, I ran various versions of this application. The variations for this model were SMOTE, standard, threshold, and class_weight: balanced. You can see the results in Table 1.

Metric	Standard	Threshold	class_weight: Balanced	SMOTE
PR-AUC	0.275	0.327	0.317	0.327
Recall 0	0.96	0.44	0.83	0.927
Recall 1	0.191	0.77	0.532	0.34
Accuracy	0.837	0.49	0.782	0.833

Table 1

While the standard decision tree model results in a high accuracy of 83%, the Recall for class 1 is poor with 19% and PR-AUC of 27%. The best variant of this model is to use **threshold**.
This model with threshold is able to reach a Recall for class 1 of 77% but with a low Recall for class 0 of 44% as seen in the Confusion Matrix below in Figure 15. Accuracy is 49% which is also quite low and a PR-AUC of 32%.

```
====================================
Chosen Threshold: 0.057
Target Recall: 0.7
====================================

Confusion Matrix (thresholded):
                  Pred: No Attr  Pred: Attr
Actual: No Attr             109         138
Actual: Attr                 11          36

ROC-AUC: 0.6885605995348436
PR-AUC: 0.3273045301295971

Classification Report:

              precision    recall  f1-score   support

           0       0.91      0.44      0.59       247
           1       0.21      0.77      0.33        47

    accuracy                           0.49       294
   macro avg       0.56      0.60      0.46       294
weighted avg       0.80      0.49      0.55       294
```

Figure 15

Feature importance for Decision Trees using weight_class balance gives the top 3 features, Total Working Years, Overtime, and Age.

```
TotalWorkingYears     0.198218
OverTime              0.166605
Age                   0.106601
StockOptionLevel      0.100886
NumCompaniesWorked    0.094860
JobLevel              0.062221
DailyRate             0.033095
Department_Sales      0.031323
WorkLifeBalance       0.030547
DistanceFromHome      0.024875
```

Figure 16

3.4.4 XGBoost

For the final model, I used XGBoost which is short for eXtreme Gradient Boosting. “It is an optimized implementation of Gradient Boosting and is a type of ensemble learning method that combines multiple weak models to form a stronger model.

XGBoost uses decision trees as its base learners and combines them sequentially to improve the model’s performance. Each new tree is trained to correct the errors made by the previous tree and this process is called boosting.
It has built-in parallel processing to train models on large datasets quickly. XGBoost also supports customizations allowing users to adjust model parameters to optimize performance based on the specific problem.” [14]. The library xgboost was used for this algorithm. [15].
While this model is able to reach a Recall for class 1 of 72%, Accuracy and Recall for class 0 suffer. Recall for class 0 is low at 58% and Accuracy of 61%. Figure17

```
====================================
Chosen Threshold: 0.011
Target Recall: 0.7
====================================

Confusion Matrix (thresholded):
                  Pred: No Attr  Pred: Attr
Actual: No Attr            144         103
Actual: Attr                13          34

ROC-AUC: 0.7383064863467999
PR-AUC: 0.47917032372615165

Classification Report:

              precision    recall  f1-score   support

           0       0.92      0.58      0.71       247
           1       0.25      0.72      0.37        47

    accuracy                           0.61       294
   macro avg       0.58      0.65      0.54       294
weighted avg       0.81      0.61      0.66       294
```

Figure 17

Feature importance for xgboost varies from the other models used so far. For the top 3 features, we get Total Working Years, Job Roles Sales Executive, and Stock Option Level. Figure 18

```
TotalWorkingYears                    0.057987
JobRole_Sales Executive              0.051324
StockOptionLevel                     0.048075
EducationField_Technical Degree      0.047660
OverTime                             0.046321
JobRole_Research Scientist           0.044655
MaritalStatus_Divorced               0.033816
EnvironmentSatisfaction              0.033344
YearsWithCurrManager                 0.032277
Age                                  0.031349
```

Figure 18

3.4.5 AdaBoost

The last model I wanted to try was AdaBoost, as it is mentioned in Maytha AL-Ali et al [2], as a top performing model. I applied various versions of imbalance solutions and threshold remained the best for this model as well. One important note is that the threshold for this model remained as close to the default of 50% when compared to all other models. Figure 19.

This model seems very promising as it is able to reach a high recall of 72% and 74% for class 1 and 0 respectively. Accuracy remains high at 73% as well with PR-AUC of 45%. This is positive result as it minimizes false positives while capturing 72% of true positives.

```
====================================
Chosen Threshold: 0.422
Target Recall: 0.7
====================================

Confusion Matrix (thresholded):
                  Pred: No Attr  Pred: Attr
Actual: No Attr            182          65
Actual: Attr                13          34

ROC-AUC: 0.799552071668533
PR-AUC: 0.44931006890543845

Classification Report:

              precision    recall  f1-score   support

           0       0.93      0.74      0.82       247
           1       0.34      0.72      0.47        47

    accuracy                           0.73       294
   macro avg       0.64      0.73      0.64       294
weighted avg       0.84      0.73      0.77       294
```

Figure 19

4. Evaluation

For each of the models, performance was evaluated using a hold-out test set. Given the imbalanced nature of the dataset (16% attrition), model comparison prioritized recall for the minority class (Attrition) to minimize false negatives. Missing at-risk employees would be more costly than incorrectly flagging non-attrition cases. Therefore, recall and PR-AUC were emphasized over accuracy.

To address class imbalance, multiple strategies were tested, including class weighting, SMOTE resampling, and threshold optimization. Threshold selection was guided by a target recall of at least 70% for the attrition class using the precision-recall curve.

Table 2 summarizes the performance of the final selected configurations for each model.

Model	**Recall - Attrition**	**PR-AUC**	**Accuracy**
Logistic Regression	72%	56.9%	73%
Random forest	72%	42.4%	78%
XGBoost	72%	47.9%	61%
AdaBoost	72%	44.5%	73%
Decision Tree	77%	32.7%	49%

Table 2

Although the Decision Tree achieved the highest recall (77%), its accuracy and PR-AUC were substantially lower, indicating poor balance between classes. Random Forest achieved strong overall accuracy but lower precision-recall performance compared to Logistic Regression. AdaBoost and Logistic Regression provided the best trade-off between recall and overall performance, achieving 72% recall with balanced accuracy (73%) and competitive PR-AUC, 56.9% for LR and 44.5% for AdaBoost.

Overall, ensemble methods demonstrated more stable and balanced performance than standalone decision trees. Threshold optimization consistently outperformed synthetic resampling (SMOTE), suggesting that adjusting classification probability cutoffs is an effective strategy for imbalanced HR datasets.

6. Discussion

The results of this study highlight several recurring insights regarding employee attrition. Across all models, the most common set of features that help predict attrition risk were overtime, total working years, job level, and years since last promotion. These findings suggest that workload intensity and career progression are strong factors that influence employee attrition.

The results from association rules analysis supports these findings as well. For example, employees who are single and work overtime were found to have 3 times higher likelihood of leaving compared to the average employee.

From the modeling results, Logis Regression had the best overall performance based on PR-AUC while maintaining competitive recall and accuracy. This result indicates that the relationship between features and attrition risk may be largely linear. Adaboost performed well also as an ensemble model, providing an alternative to identify non linear interactions.

Regarding class imbalance, threshold optimization consistently outperformed synthetic resampling. This suggests that adjusting the decision boundary is an effective strategy for addressing class imbalance.

The findings of this study can be helpful for organizations seeking to reduce employee turnover. Predictive models can support proactive retention strategies by identifying high-risk employees early, enabling targeted interventions such as workload adjustments, career development opportunities, or engagement initiatives.

Despite these contributions, several limitations should be noted. The dataset used represents a

relatively small sample and may not capture broader organizational or industry variations. Additionally, the data provides a static snapshot rather than longitudinal information, limiting the ability to model temporal patterns of attrition. Future work could incorporate larger datasets, further feature selection methods to minimize over fitting, time-series analysis, and explainable AI techniques such as SHAP to further enhance model interpretability and decision support.

6.1 Timeline

Work timeline during 4 weeks

Week 1: Data Understanding - Complete

- Review and understand the data
- Review all features to ensure proper understanding of its values
- Review dataset size and data types

Week 2: Data Preparation - Complete

- Review data for completeness and consistency
- Feature engineering
- Class imbalance
- Training and test splits

Week 3: Modeling - Complete

- Train and test various models
- Test models with various solutions for class imbalance
- Determine best class imbalance solution for model type and pick the best one

Week 4: Evaluation - Complete

- Review results from each model
- Compare results of every model
- Pick the best model and evaluate its results

7. Conclusion

This study investigated the application of data mining and machine learning techniques to predict employee attrition using a Human Resources dataset containing 1,470 employee records and multiple organizational features. The analysis included comprehensive data preprocessing, feature engineering, class imbalance handling, and evaluation of both unsupervised and supervised learning methods.

Multiple classification models were implemented and compared, including Logistic Regression, Decision Trees, Random Forest, XGBoost, and AdaBoost. Special emphasis was placed on addressing class imbalance through threshold optimization, class weighting, and SMOTE resampling. Among these approaches, threshold adjustment consistently provided the most effective balance between recall and overall model performance.

Logistic Regression achieved the strongest overall results, demonstrating the highest PR-AUC while maintaining competitive recall and accuracy. Ensemble models such as Random Forest and AdaBoost also showed stable performance, indicating their effectiveness in handling complex feature interactions. Across models, key predictors of attrition consistently included overtime, total working years, job level, and promotion history.

These findings demonstrate that machine learning models can effectively identify employees at risk of attrition and provide actionable insights for workforce management. By enabling early detection of high-risk employees, predictive analytics can support targeted retention strategies and improve organizational decision-making

8. References

[1] Saurabh Badole. 2024. HR Employee Attrition. Retrieved on Jan 2026. https://www.kaggle.com/datasets/saurabhbadole/hr-employee-attrition/data

[2] Maytha AL-Ali. 2026. Integrating machine learning and explainable AI for employee attrition prediction in HR analytics. https://pmc.ncbi.nlm.nih.gov/articles/PMC12905426/#Sec2

[3] Sheik Aman. Oct 2025. Evaluation Metrics in Machine Learning. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/

[4] Emma Ruel. 2024. Attrition: Definition, Types, Causes & Mitigation Tips. https://www.shrm.org/topics-tools/news/employee-relations/attrition-definition-types-causes-mitigation-tipsApril

[5] Lekhana G. 2025. One Hot Encoding in Machine Learning. https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/

[6] MLExtend Documentation. apriori: Frequent itemsets via the Apriori algorithm. https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/apriori/

[7] MLExtend Documentation. Association Rules. association_rules: Association rules generation from frequent itemsets. https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/

[8] Yotam Elore. May 2022. To SMOTE, or not to SMOTE?. https://arxiv.org/abs/2201.08528

[9] SMOTE documentation. https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html

[10] Logistic Regression Documentation. LogisticRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[11] Precision Recall Curve documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_curve.html

[12] Susmit Sekhar Bhakta. Dec 2025. Random Forest Algorithm in Machine Learning. https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

[13] Decision Tree documentation. Decision Trees. https://scikit-learn.org/stable/modules/tree.html

[14] Pawan Saxena. Oct 2025. XGBOOST. https://www.geeksforgeeks.org/machine-learning/xgboost/

[15] Xgboost. Xgboost documentation. https://xgboost.readthedocs.io/en/stable/index.html

[16]